The Journal of Supercomputing (2026) 82:402
https://doi.org/10.1007/s11227-026-08533-2

A reduction framework with an improved beam search algorithm for non-slicing VLSI floorplanning

Canhui Luo[1] · Yaozhong Zhao[1] · Yan Li[1] · Zhouxing Su[1] · Qingyun Zhang[1] · Junwen Ding[1] · Zhipeng Lü[1]

Received: 24 October 2025 / Accepted: 10 April 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract
Floorplanning is one of the most critical steps in the very large-scale integration (VLSI) physical design flow. It directly influences the performance, power, and area of high-performance chips. In computational science, floorplanning is a widely studied NP-hard problem. It requires generating a non-overlapping placement, i.e., a floorplan, for a set of circuit blocks interconnected by nets, each of which connects a subset of the blocks. The objective is to minimize the bounding box area of the floorplan and the total half-perimeter wirelength (HPWL) of the nets. This paper presents a reduction framework for solving the floorplanning problem, which reformulates it into a series of fixed-width floorplanning subproblems for systematic exploration and optimization. To solve each subproblem, we propose an improved beam search algorithm (IBSA) that integrates a heuristic construction guided by a combined ranking of area and wirelength scores. To balance local exploitation and global exploration in the beam search process, we adopt an adaptive hierarchical evaluation paradigm where the granularity of evaluation varies from local, look-ahead, to global views. Extensive experiments on the well-studied MCNC and GSRC benchmark suites show that, in the area-only setting, IBSA establishes new best-known upper bounds in terms of floorplan area. Under joint optimization of area and wirelength, IBSA produces smaller areas at comparable HPWL than the state-of-the-art floorplanners. Moreover, IBSA shows competitive runtime performance on the tested instances.

Keywords VLSI floorplanning · Fixed-width floorplanning · Improved beam search · Area and wirelength optimization · Adaptive hierarchical evaluation

✉ Zhouxing Su
suzhouxing@hust.edu.cn

Canhui Luo
luocanhui@hust.edu.cn

Zhipeng Lü
zhipeng.lv@hust.edu.cn

[1] School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

Published online: 08 May 2026

1 Introduction

Floorplanning is a critical step in the modern physical design flow of VLSI circuits. It aims to determine the relative positions and orientations of circuit components on a chip to meet various design constraints. Typically, floorplanning optimizes two key objectives, namely minimizing the bounding box area of all components and the total wirelength of the netlist. An effective floorplan influences performance, power, and area, and it affects the manufacturability of high-performance chips [1, 2]. As semiconductor technology advances and design complexity increases, floorplanning becomes increasingly challenging. In particular, tighter area budgets and stricter outline constraints make it harder to achieve feasible and high-quality floorplans.

Over the years, floorplanning has attracted significant attention from the research community. The solution representations for the floorplanning problem can be slicing and non-slicing [3]. Slicing floorplanning divides the chip area recursively into rectangular subareas, called rooms, using horizontal and vertical guillotine cuts. Each room accommodates a circuit component, forming a complete floorplan. Despite its efficiency in solving the problem, this method may fail to represent the optimal floorplan, particularly in area utilization [4]. In contrast, non-slicing floorplanning employs a more flexible configuration, allowing blocks to be placed at arbitrary positions and orientations within the chip area, thus offering greater adaptability to mixed-size components and wirelength optimization. Therefore, non-slicing floorplanning is more common in practical applications than slicing floorplanning. In practical applications, floorplanning problems can be categorized into free-outline and fixed-outline floorplanning [5]. Free-outline floorplanning provides enhanced flexibility in minimizing area and wirelength during the early design. However, the generated floorplans may not fit into fixed-size dies. In contrast, fixed-outline floorplanning requires all modules to be placed within a predefined boundary, aligning with the layout optimization constraints of the physical design phase. The latter poses greater optimization complexity, particularly under tight area constraints imposed by the fixed boundary. Given their respective significance in different application scenarios, this study focuses on optimizing non-slicing floorplanning for both free-outline and fixed-outline floorplanning problems.

Due to the geometric constraints inherent in floorplanning, solution representation affects optimization efficiency and is closely related to the methods employed. Typically, slicing floorplanning employs a slicing tree to represent a floorplan. This representation has enabled a series of effective algorithms [6, 7]. For non-slicing floorplanning, Guo et al. [8] devised the representation of the O-tree and proposed a deterministic construction algorithm. Chang et al. [9] introduced the B*-tree structure, which enables fast operations such as swapping and rotating blocks, and proposed a simulated annealing (SA) algorithm for floorplanning. Building on this, Lin et al. [10] proposed the skewed B*-tree (SKB-tree) to accommodate fixed-outline floorplanning constraints better. In addition to tree structures, Ma et al. [11] and Lin et al. [12] proposed SA processes based on corner block list (CBL) and transitive closure graph (TCG), respectively. Tang et al. [13] introduced the sequence pair (SP) to guide a constructive algorithm to represent a floorplan. Gwee et al. [14] proposed an inte-

ger coding representation and heuristic-based decoder for floorplans that facilitates optimization using genetic algorithms (GAs).

Building upon classical layout representations, recent research has advanced floorplanning optimization through a broad spectrum of algorithmic paradigms. Simulated annealing (SA) remains a foundational method, with variants such as SA-EA [15], ASSA [16], HSA [17], and ISSAA [18], enhancing performance through specialized penalty functions, hybridization strategies, and customized cooling schedules, respectively. In parallel, evolutionary and swarm intelligence methods have also been extensively explored, including genetic algorithms [19] and particle swarm optimization (PSO) in both standard [20] and discrete multi-objective forms (DPSO) [21]. The PSO-GA based hybrid algorithm (PGHA) [22] combines both GA and PSO for thermal-aware non-slicing floorplanning. Hybrid memetic methods, such as the adaptive hybrid memetic algorithms AHMA [23] and DPAHMA [24], and the evolutionary memetic algorithm (EMA) [25] integrate memetic strategies with simulated annealing-based local search to balance global diversification and local intensification. In addition, nature-inspired metaheuristics including the variable-order ant system (VOAS) [26] and the lion optimization algorithm (LOA) [27], together with constructive heuristics such as deferred decision-making (DeFer) [28], have further enriched the methodological repertoire. More recently, reinforcement learning (RL) has been applied to floorplanning either to autonomously learn effective local search heuristics [29] or in combination with genetic algorithms to guide evolutionary search (GARL) [30]. Although RL remains less mature in this domain, these efforts suggest its potential for improving both construction and optimization processes.

Although these approaches achieve competitive area and wirelength results, they mostly rely on discrete encodings of geometric layouts. Such representations do not naturally accommodate hard geometric constraints, particularly in fixed-outline floorplanning. Consequently, many methods introduce penalty terms to discourage boundary violations, including SA-EA [15] and EMA [25]. However, outline feasibility is highly sensitive to local perturbations. Even minor moves, such as swapping or rotating a block, can induce a discontinuous change in the extent of boundary violation and may flip feasibility. This behavior makes it nontrivial to design penalty functions that remain both effective and robust across instances, especially under tight outline constraints. Moreover, some memetic frameworks, such as AHMA [23] and DPAHMA [24], do not explicitly account for fixed-outline feasibility during optimization, which limits their applicability when strict boundary satisfaction is required.

Motivated by this gap, we revisit floorplanning from the perspective of area minimization and its close connection to the two-dimensional rectangle packing area minimization problem (RPAMP). Upon considering wirelength minimization as an additional objective, various effective techniques and foundational tools proposed for RPAMP can also be applied to optimize floorplanning. Notably, the work by Imahori et al. [31] has already generated RPAMP instances based on the floorplanning benchmarks MCNC and GSRC, focusing only on minimizing area. Constructive algorithms for RPAMP typically employ corner occupancy [32] and best-fit strategies [33]. Burke et al. [34] introduced the skyline data structure to efficiently maintain

accessible areas during the block placement process. Based on efficient constructive algorithms, He et al. [35] presented a dynamic reduction algorithm that reduces the RPAMP to multiple rectangle packing problems. Recently, Wei et al. [36] proposed an adaptive selection approach that decomposes the original problem into a series of strip-packing subproblems and strategically employs a random local search for optimization. In addition, when floorplanning is cast as a sequential decision-making problem, beam search [37] provides a natural tree-based framework for constructive exploration. These studies provide practical optimization ideas for our work on floorplanning.

This paper proposes a reduction framework for the non-slicing floorplanning problem. Previous methods usually optimize layout planning in both horizontal and vertical spatial dimensions. In contrast, we fix the horizontal dimension of the bounding box to different widths and then apply beam search along the other dimension to construct and optimize the floorplan. The reduction decomposes the original solution space into a set of width-specific subspaces, enabling guided exploration based on prior search outcomes. More importantly, this paradigm enables direct selection of an appropriate layout width to satisfy horizontal boundary constraints, while restricting the maximum layout height during optimization to ensure that the floorplan complies with the fixed-outline constraints.

Our main contributions are as follows:

- We transform the floorplanning problem into a series of subproblems, each of which aims to minimize the layout height and wirelength under a fixed width, called fixed-width floorplanning. An adaptive width selection algorithm is applied to optimize the subproblems corresponding to the promising candidate widths systematically.
- We propose a candidate width generation algorithm that produces high-quality candidate widths based on the frequency of combined widths and aspect ratio control, considering both area and wirelength optimization.
- We propose an improved beam search algorithm (IBSA) that integrates local evaluation, global evaluation, and a novel look-ahead evaluation technique to filter out unpromising branches of the search tree.
- We conduct experiments to compare the proposed algorithm with the state-of-the-art algorithms on the well-studied benchmark suites MCNC and GSRC, improving the best-known results for free-outline and fixed-outline floorplanning on the majority of the tested instances.

The remainder of the paper is organized as follows. Section 2 formulates the floorplanning problem and its transformation. Section 3 presents the proposed solution framework, including the candidate width generation algorithm and the adaptive exploration strategy. Section 4 gives a detailed description of the improved beam search algorithm for solving fixed-width floorplanning subproblems. Section 5 reports the experimental results, and Sect. 6 concludes the paper.

2 Preliminaries

2.1 Problem description

In general, the floorplanning problem involves assigning locations to a set of rectangular modules while optimizing a specified objective function. Given a set B of m blocks, each block $b_i \in B$ has a width w_i and a height h_i. Given a set N of n nets, where each net $\mu \in N$ involves a subset of blocks $B_\mu \subseteq B$ and any pair of blocks in B_μ is interconnected. The solution of the problem can be represented by a list of tuples $((x_i^-, y_i^-), d_i)$ for each block b_i, where (x_i^-, y_i^-) are the bottom-left coordinates of block b_i and $d_i \in \{0, 1\}$ indicates whether block b_i is rotated by 90°. Then, the upper-right coordinate of block b_i can be expressed as (x_i^+, y_i^+), where $x_i^+ = x_i^- + (w_i(1 - d_i) + h_i d_i)$ and $y_i^+ = y_i^- + (h_i(1 - d_i) + w_i d_i)$. Let auxiliary variables W and H denote the width and height of the bounding box. A feasible solution must satisfy all the following constraints:

$$0 \le x_i^-, x_i^+ \le W, 0 \le y_i^-, y_i^+ \le H, \forall b_i \in B \tag{1}$$

$$\max\{x_i^- - x_j^+, x_j^- - x_i^+\} \ge 0, \forall b_i, b_j \in B, b_i \ne b_j \tag{2}$$

$$\max\{y_i^- - y_j^+, y_j^- - y_i^+\} \ge 0, \forall b_i, b_j \in B, b_i \ne b_j \tag{3}$$

Constraints (1) ensure that all blocks must be placed inside the bounding box. Constraints (2) and (3) ensure that no overlap exists between any pair of blocks. The floorplanning problem is a bi-objective optimization problem whose objective functions involve both area and wirelength minimization. The first optimization objective is to minimize the area $A(F)$ of the bounding box of the floorplan F, as shown in Eq. (4).

$$\min_F \quad A(F) = W \times H \tag{4}$$

Let $A(B) = \sum_{b_i \in B} w_i \cdot h_i$ represent the total area of all blocks. In fixed-outline floorplanning, given the aspect ratio R and the maximum deadspace ratio D ($D \ge 0$), the maximum width W_{fixed} and maximum height H_{fixed} of the layout region are determined by Eqs. (5) and (6), respectively.

$$W_{\text{fixed}} = \sqrt{A(B) \cdot (1 + D) \cdot R} \tag{5}$$

$$H_{\text{fixed}} = \sqrt{A(B) \cdot (1 + D)/R} \tag{6}$$

Subsequently, a feasible fixed-outline floorplanning solution must also satisfy the outline constraint specified in Eq. (7).

$$W \le W_{\text{fixed}}, H \le H_{\text{fixed}} \tag{7}$$

The second optimization objective is to minimize the total interconnect wirelength, measured by the half-perimeter wirelength (HPWL). Specifically, the HPWL of a net is defined as the sum of the width and height of the minimum bounding rectangle enclosing the geometric centers (x_i^c, y_i^c) of each connected block b_i. The total wirelength $L(F)$ of floorplan F can be calculated by Eq. (8).

$$\min_{F} \quad L(F) = \sum_{\mu \in N} \left(\max_{b_i, b_j \in B_\mu} |x_i^c - x_j^c| + \max_{b_i, b_j \in B_\mu} |y_i^c - y_j^c| \right) \tag{8}$$

The weighted sum method is a classic technique for solving multi-objective optimization problems. A common weighted objective function has been widely adopted for the floorplanning problem in previous studies [18, 38], as follows:

$$\min_{F} Cost(F) = \xi \cdot \frac{A(F)}{A^*} + (1 - \xi) \cdot \frac{L(F)}{L^*}, 0 \leq \xi \leq 1 \tag{9}$$

Equation (9) normalizes the area and wirelength objectives and balances their optimization via the weight parameter ξ. A larger ξ favors a smaller chip area, whereas a smaller value emphasizes wirelength optimization. Typically, ξ is set to 0.5 to achieve a balanced trade-off. A^* and L^* are usually the sampled objective values, such as the average area and average wirelength of 1000 randomly generated floorplans [38], respectively. This paper adopts the algorithm proposed in Sect. 4 to rapidly estimate A^* and L^* instead of relying on random solutions. The motivation is that A^* and L^* are intended to approximate achievable lower bounds. Using optimized estimates makes the normalization more meaningful and renders the weight parameter ξ more practically interpretable in the bi-objective setting. Furthermore, when estimating A^* (or L^*), we only consider the minimization of area (or wirelength) to evaluate the lower bounds more precisely.

2.2 Problem transformation

For previous search approaches, such as the B*-tree-based simulated annealing algorithm [9], the performance can only be known after executing modification operations and reconstructing the floorplan. Due to the absence of an evaluation of partial states, the search struggles to recognize that better results might be attainable simply by refining local structures of the layout. With this in mind, we reformulate the original problem into a series of subproblems with a fixed width. The area minimization objective (4) is transformed into (10), where W_c denotes the set of candidate widths.

$$\min_{F|_{W \in W_c}} \quad A(F) = W \times H \tag{10}$$

When the width of the bounding box W for floorplanning is given, objective (4) is equivalent to minimizing the height H, while the wirelength objective remains unchanged. We name it fixed-width floorplanning and place blocks upwards sequentially to build a complete floorplan. The fixed-width constraint enables efficient

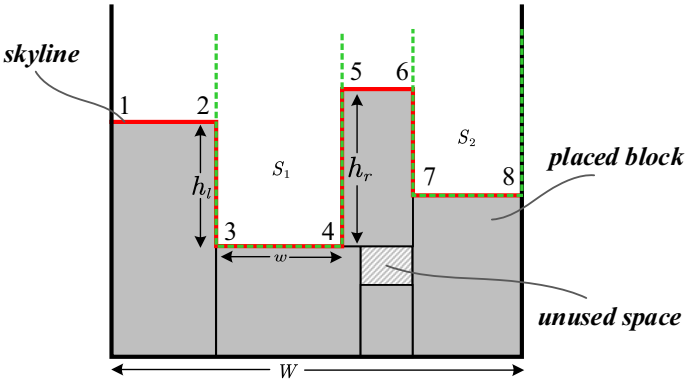

Fig. 1 An example of a skyline, along with two spaces, s_1 and s_2

maintenance and evaluation of the shapes of already placed blocks. This allows for careful consideration of either the matching pattern of the next block (local evaluation) or the overall performance (global evaluation), thereby effectively optimizing the layout height. Additionally, the mutual independence of the subproblems naturally provides potential for parallel optimization of the overall problem. The associated challenge is generating promising candidate widths and efficiently exploring the optimization of the corresponding fixed-width floorplanning subproblems. Based on this idea, we propose a novel framework for solving floorplanning in Sect. 3 and describe our heuristic for fixed-width floorplanning in Sect. 4.

2.3 Skyline heuristic

The representation of a floorplan is crucial in algorithm design, as it affects computational efficiency and is closely linked to optimization strategies. We employ a data structure called the *skyline* to represent the contour of the partially placed blocks in a floorplan. The skyline is an efficient data structure, originally proposed for the rectangular packing problem [39]. It consists of several horizontal segments that maintain the front boundary of the region occupied by placed blocks and unused space. Each segment is determined by a left endpoint (x, y) and a segment width w. Specifically, in the initial state, the skyline consists of a single segment starting at $(0, 0)$ with a width W specified by the fixed-width floorplanning problem. A segment with its nearest left and right vertical boundaries forms a *space*. Based on the skyline, we can easily find the bottom-left accessible space to place the block and update the skyline in constant time. Figure 1 illustrates an example of a skyline and its corresponding spaces. The solid red line indicates the skyline, while spaces s_1 and s_2 are represented by dashed green lines, where the heights of the left and right boundaries of s_1 are h_l and h_r, respectively. Generally, we choose the endpoints of the segment of the bottom-left space as candidate placement points, such as endpoints 3 and 4 of space s_1. Subsequently, the next block is selected using heuristic rules that jointly consider the matching degree

between candidate blocks and the bottom-left space, as well as the interconnection wirelength to already placed blocks.

3 A new framework for solving floorplanning

We propose a general framework for solving floorplanning, consisting of two components: (1) Generating a set of promising candidate widths for optimization, which transforms the original problem into a series of fixed-width floorplanning subproblems; (2) Adaptively and iteratively selecting widths from the candidate set and employing a fixed-width floorplanning solver for optimization.

3.1 Candidate width generation

In principle, every possible combination of block edge lengths can yield a feasible candidate width. For each block $b \in B$, let l_b and s_b denote the longer and shorter edges of its bounding box, respectively. The valid range of candidate widths is defined as $[W_\mathrm{l}, W_\mathrm{u}]$, where $W_\mathrm{l} = \max_{b \in B} s_b$ and $W_\mathrm{u} = \sum_{b \in B} l_b$. Since this interval can be excessively wide, solving fixed-width floorplanning problems for all widths within it would result in significant redundant computation. In the literature, Bortfeldt [40] and Wei et al. [36] proposed three generation strategies, namely WV1, WV2, and WV3, to produce a smaller yet promising subset of candidate widths.

The first approach WV1 narrows the search space by redefining the upper bound as $W_\mathrm{u} = \lfloor \beta \sqrt{A(B)} \rfloor$, where $\beta > 1$ and $A(B)$ represent the total area of the block set. Then, n_w candidate widths are uniformly sampled from $[W_\mathrm{l}, W_\mathrm{u}]$. Although it is simple and efficient, the uniform sampling strategy may overlook promising widths, as it ignores the quality of individual configurations.

To address this issue, WV2 introduces an evaluation-based mechanism. A lightweight subroutine that produces medium-quality layouts in very short runtimes is employed to assess each potential width in $[W_\mathrm{l}, W_\mathrm{u}]$. The top n_w widths that yield the smallest bounding boxes are retained as candidates. Typical implementations of this subroutine include constructive heuristics such as BFDH* [40] or random local search methods [36]. This refinement improves selection accuracy but comes at the expense of additional evaluation effort.

A further alternative WV3 estimates the potential of a width by counting the number of dimension combinations that yield the same total width, thereby avoiding reliance on an evaluator and the associated computational overhead. Specifically, each time a set of block edges adds up to a particular width, the counter for that width is increased. The n_w widths with the highest counts are then selected as candidates. This frequency-based scheme assumes that widths with more combinations are more likely to yield compact layouts, and has been shown to perform effectively in practical optimization [36].

Although the aforementioned strategies are effective, they were specifically designed for the 2D rectangle packing area minimization problem and therefore focus only on area optimization without accounting for aspect ratio or wirelength, both of which are essential in floorplanning optimization. Motivated by these limitations, we

Algorithm 1 Adaptive Width Selection Algorithm

Require: A set of input blocks B, a set of candidate widths W_c.
Ensure: F^*: the best floorplan found.
$F^* \leftarrow \varnothing, O^* \leftarrow \infty$ ▷ O^* denotes the objective value of F^*
for each candidate width $W_i \in W_c$ **do**
  $\gamma_i \leftarrow 1$
  $O_i, F_i \leftarrow IBSA(B, W_i, \gamma_i)$
  if F_i is feasible and $O_i < O^*$ **then**
    $O^* \leftarrow O_i, F^* \leftarrow F_i$
  end if
end for
Sort $W_i \in W_c$ in a descending order of O_i
for $i = 1$ to $|W_c|$ **do**
  $P_i \leftarrow \frac{2 \cdot i}{|W_c| \cdot (|W_c|+1)}$
end for
while the stop condition is not met **do**
  Randomly select W_i with probability P_i
  $\gamma_i \leftarrow min(2 \cdot \gamma_i, \gamma_u)$
  $O^{'}, F^{'} \leftarrow IBSA(B, W_i, \gamma_i)$
  if $F^{'}$ is feasible and $O^{'} < O_i$ **then**
    $O_i \leftarrow O^{'}$
    if $O_i < O^*$ **then**
      $O^* \leftarrow O^{'}, F^* \leftarrow F^{'}$
    end if
    Sort $W_i \in W_c$ in a descending order of O_i and update P_i for each W_i
  end if
end while
return F^*

propose CWGA, a refined candidate width generation algorithm that simultaneously considers area and wirelength within a unified framework. We first restrict the candidate range by introducing two parameters, ρ and τ, to control the aspect ratio of the floorplan. Specifically, the lower and upper bounds are redefined as $W_l = \lfloor \rho \sqrt{A(B)} \rfloor$ and $W_u = \lfloor \tau \sqrt{A(B)} \rfloor$, where $0 < \rho < 1$ and $\tau \geq 1$. A near-unity aspect ratio (≈ 1.0) is often advantageous for wirelength minimization since it allows for tighter clustering of blocks. In fixed-outline floorplanning, there are additional constraints, specifically $W_u \leq W_{\text{fixed}}$ and $W_l \geq \lceil \frac{A(B)}{H_{\text{fixed}}} \rceil$, to eliminate widths that are infeasible. Subsequently, we apply the aforementioned WV3 method within the interval $[W_l, W_u]$ to select the top n_w combination widths with the highest frequency as candidate widths. A special case arises when the total number of possible width combinations is less than n_w, with all combinations selected. By integrating these two steps, our CWGA considers candidate widths that are simultaneously beneficial for both area and wirelength optimization while avoiding a large candidate set.

3.2 Adaptive width selection and optimization

Given a set B of blocks and a set W_c of candidate widths, we employ an adaptive width selection approach (AWSA) to systematically explore optimization across different widths. The detailed procedure is described in Algorithm 1. The core idea is to select

the width probabilistically based on prior results of the corresponding fixed-width floorplanning.

We denote the solver of floorplanning under a fixed width W as IBSA(B, W, γ), where parameter γ controls the search depth. AWSA first calls IBSA to solve each candidate width $W_i \in W_c$ with a small γ (typically $\gamma = 1$) to obtain the initial objective value (lines 2–8). Then, all candidate widths W_i are sorted in descending order by their objective values O_i (line 9). Note that for free-outline floorplanning, IBSA always returns feasible solutions, whereas for fixed-outline floorplanning, it is necessary to check whether the layout height satisfies the H_{fixed} constraint (lines 5 and 17). In each subsequent iteration, AWSA selects the ith-ranked width w_i for optimization with probability $\frac{2 \cdot i}{|W_c| \cdot (|W_c|+1)}$ (lines 11 and 14). For instance, when the candidate set contains four widths, the probabilities range from 0.1 for the lowest-ranked width to 0.4 for the highest-ranked one. IBSA(B, W_i, γ_i) is called for optimization of the chosen width, and the value of γ_i is set to double the value when W_i was last selected, allowing for a deeper search (lines 15–16). Parameter γ_i is limited to no more than γ_u to avoid spending too much effort on the same width. Upon updating the optimal floorplan for any width W_i, all candidate widths are re-ranked using the new objective values (lines 18–22). This strategy prefers to select the widths with high potential for improving subsequent iterations, and the less promising widths are still chosen with a low probability rather than being discarded. The search process terminates when the time limit is reached, and the best solution found so far is returned.

AWSA with CWGA serves as a general framework for floorplanning. In this paper, IBSA is proposed as the fixed-width floorplanning solver within the AWSA procedure. It can be replaced by other algorithms, and the optimization depth, determined by the parameter γ, can also be adjusted using metrics such as search time.

4 Improved beam search algorithm for fixed-width floorplanning

The beam search algorithm (BSA) is a best-first tree search method with aggressive pruning. BSA employs a greedy algorithm to evaluate the upper bound of branches and always selects the branch with the best upper bound without backtracking. The greedy evaluation is typically further decomposed into local and global evaluation strategies to balance efficiency and effectiveness. The former rapidly evaluates and reduces candidate branches from a local view, and the latter explores promising branches identified by a global view.

4.1 Improved beam search algorithm framework

This paper proposes an improved BSA (IBSA) framework, as illustrated in Fig. 2. It integrates a novel look-ahead evaluation pruning technique and optimizes the control of the pruning parameters. The overall procedure can be summarized in the following four steps:

1. Generating candidate branches. In the search tree, each non-leaf node corresponds to a partial floorplan, and each unplaced block is added to it to generate a new child

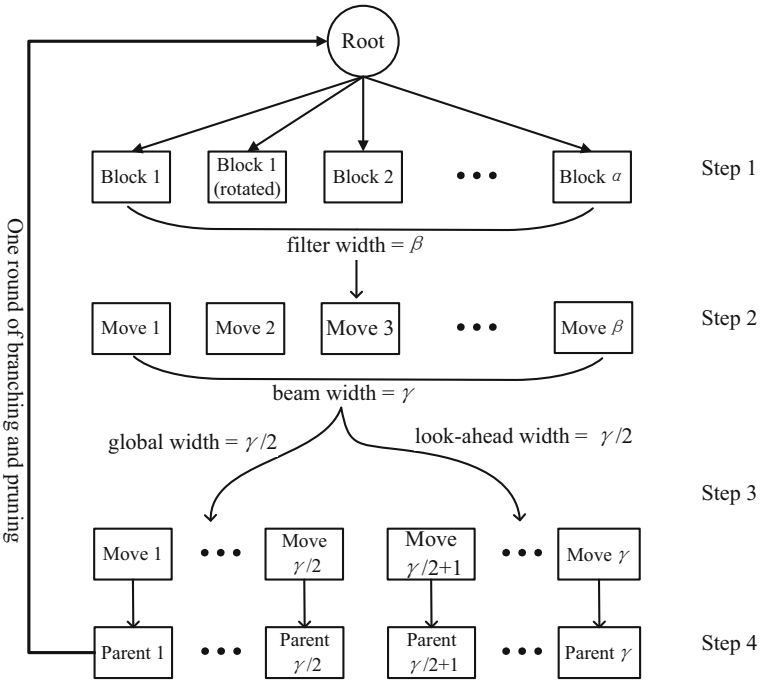

Fig. 2 The framework of the improved beam search algorithm

node, with the total number of candidate branches limited by the branch width parameter α.

2. Pruning branches according to local evaluation. The local evaluation, based on changes in the skyline and wirelength, retains only the most promising child nodes for the next step, with the number of retained nodes controlled by the filter width parameter β.
3. Performing global evaluation on the retained nodes. The partial solution corresponding to each child node is completed using the global evaluation strategy to obtain an upper bound. To improve accuracy while reducing overhead, a look-ahead evaluation strategy may terminate early. These two strategies are applied to γ nodes, where γ is the beam width parameter.
4. Expanding the best branches. The best child nodes from the global (or look-ahead) evaluation are selected to form new parent nodes, and branching and pruning are repeated until all blocks are placed or the time limit is reached. Finally, the best solution is returned among all complete and feasible solutions obtained.

We apply IBSA to solve the fixed-width floorplanning problem. Algorithm 2 gives the pseudo-code of IBSA. Given a block set B, a bounding box width W, and a beam width γ, IBSA returns the best objective function value O^* found and the corresponding floorplan F^*. The search tree grows layer by layer based on the branching and pruning strategies until all blocks are placed. In each search layer, we first branch on all current parent nodes T by generating child nodes from their unplaced blocks,

thereby forming the complete set of candidate child nodes (lines 5–8). The detailed branching procedure is provided in Sect. 4.2. Subsequently, three pruning strategies are applied to evaluate and filter these candidates: local evaluation, global evaluation, and look-ahead evaluation. The local evaluation conducts a rapid fitness assessment, combining area-based scoring with wirelength estimation, and retains the top $\beta = 2\gamma$ nodes (lines 9–12). The global evaluation extends these nodes into complete solutions to obtain reliable upper bounds, preserving the best $\lceil \frac{\gamma}{2} \rceil$ nodes (lines 13–19). Since constructing complete solutions becomes less reliable when many blocks remain unplaced due to the long-term greedy effect, a look-ahead evaluation is introduced (lines 20–27). It performs partial forward floorplanning over a limited horizon, striking a balance between short- and long-term accuracy. Similar to the global evaluation, the look-ahead evaluation also retains the best $\lfloor \frac{\gamma}{2} \rfloor$ nodes. These three strategies will be elaborated in Sect. 4.3. Finally, the γ pruned child nodes serve as the new parent nodes for the next layer of the tree search (line 28).

Traditional beam search algorithm takes three hyperparameters: branch width α, filter width β, and beam width γ. Based on preliminary experiments, the parameters of our algorithm are set as follows: (1) Branch width α equals the number of child nodes since it is unreasonable to discard child nodes without evaluation. (2) Filter width β and beam width γ satisfy $\beta = 2\gamma$ so that the placements with large local waste can be abandoned without heavy evaluation (lines 3 and 12). Therefore, IBSA only accepts beam width γ as a hyperparameter. When beam width $\gamma = 1$, the search tree degenerates to a single chain, and only the best child node is retained in each layer. In this case, IBSA is equivalent to a fast and coarse-grained greedy algorithm. The width of the search tree expands as γ increases, resulting in a larger search space. It can enhance the quality of IBSA solutions, albeit accompanied by increased computational time. In the case of a sufficiently large γ, IBSA deteriorates to a time-consuming and fine-grained exact algorithm.

4.2 Branching strategy

The critical decision in each step of floorplanning optimization involves selecting an appropriate block and determining its position. We adopt the principle of skyline-based placement (see Sect. 2.3) to simplify the decision-making. Algorithm 3 presents the pseudo-code of the branching strategy in IBSA. Specifically, for a given tree node (i.e., partial solution), IBSA considers only the lowest segment of its skyline as the candidate placement space s_{bl} (line 1). Then, IBSA branches on the selection of the next block, considering each unplaced block and its rotation (lines 4–8). For each move that fits into the candidate space s_{bl}, a corresponding child node is generated (lines 9–12). In this process, the skyline segment determines the height at which a block is placed, while the subsequent local evaluation strategy determines its specific horizontal position.

Algorithm 2 Improved Beam Search Algorithm

Require: A blocks set B, a box width W, and a beam width γ.
Ensure: The minimum objective O^* found and the corresponding floorplan F^*.

$T \leftarrow \{root \leftarrow \{B, F \leftarrow \varnothing\}\}$
$O^* \leftarrow \infty, F^* \leftarrow \varnothing$
$\beta \leftarrow 2 \cdot \gamma$
for $layer \leftarrow 1$ **to** $|B|$ **do**
  The set of child nodes $C \leftarrow \varnothing$
  for each parent node $p_i \in T$ **do**
    $C \leftarrow C \cup \{Branch(p_i)\}$
  end for
  for each child node $c_i \in C$ **do**
    $l_i \leftarrow LocalEvaluation(c_i)$
  end for
  $C \leftarrow$ Retain the top β child nodes with minimum l_i
  for each child node $c_j \in C$ **do**
    $g_j, f_j \leftarrow GlobalEvaluation(W, c_j)$
    if $g_j < O^*$ **then**
      $O^* \leftarrow g_j, F^* \leftarrow f_j$
    end if
  end for
  $C_g \leftarrow$ Select the top $\lceil \frac{\gamma}{2} \rceil$ child nodes with minimum g_j from C
  $C \leftarrow C \setminus C_g$
  for each child node $c_k \in C$ **do**
    $a_k, f_k \leftarrow LookaheadEvaluation(W, c_k)$
    if f_k is complete and $a_k < O_{best}$ **then**
      $O^* \leftarrow a_k, F^* \leftarrow f_k$
    end if
  end for
  $C_a \leftarrow$ Select the top $\lfloor \frac{\gamma}{2} \rfloor$ child nodes with minimum a_k from C
  $T \leftarrow C_g \cup C_a$
end for
return (O^*, F^*)

Algorithm 3 Branching Strategy In IBSA

Require: A partial floorplan *parent*, consisting of the current placement pattern and the set of unplaced blocks.
Ensure: The set of child nodes derived from *parent*.

$s_{bl} \leftarrow$the unfilled bottom-left space of *parent*
$B_u \leftarrow$ the set of unplaced blocks that fit in s_{bl}
children $\leftarrow \varnothing$
for each block $b_i \in B_u$ **do**
  for *rotation* $= 0$ to 1 **do**
    if *rotation* $= 1$ **then**
      swap the width and height of b_i
    end if
    if block b_i is fit into s_{bl} **then**
      child $\leftarrow$ put block b_i to the space s_{bl}
      children $\leftarrow$ *children* $\cup$ {*child*}
    end if
  end for
end for
return *children*

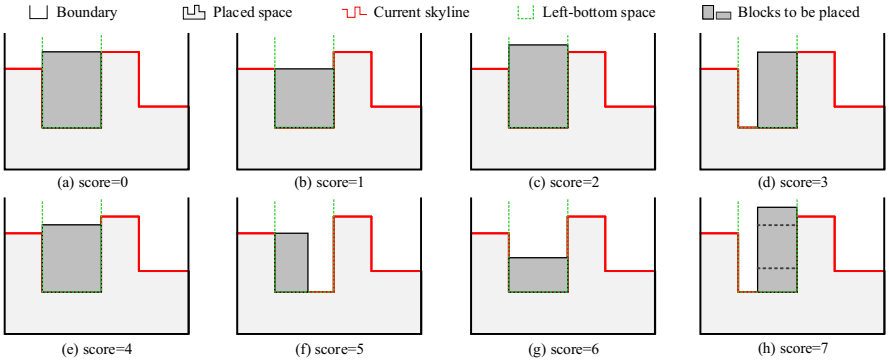

Fig. 3 Area scoring strategies for rectangular block placement in a candidate space when $h_{\mathrm{l}} < h_{\mathrm{r}}$. Each subfigure illustrates a placement pattern together with its corresponding area score. In subfigure (h), the dashed outlines indicate blocks with three different relative heights

4.3 Pruning strategy

In this section, we present our pruning strategy, which consists of three hierarchical estimations, namely local evaluation, global evaluation, and look-ahead evaluation, to make a trade-off between the effectiveness and efficiency of IBSA.

4.3.1 Local evaluation

The local evaluation strategy is to score the candidate branches based on the geometrical pattern of the current partial solution, which involves the evaluation of the local area and local wirelength.

We adopt the scoring strategy proposed by Wei et al. [36] for the rectangular packing problem to evaluate local area fitness, aiming to flatten the skyline as much as possible after placing each block. Assuming that the height of the left (right) wall of the bottom-left space is h_{l} (h_{r}), then there are eight cases when $h_{\mathrm{l}} < h_{\mathrm{r}}$, as shown in Fig. 3. Each case is assigned a distinct score between 0 and 7, which is used to differentiate the placement priorities of blocks for a specific space. Conversely, the other eight cases, symmetric to the previous ones, occur when $h_{\mathrm{l}} \geq h_{\mathrm{r}}$. The scoring values for each case in the original strategy have been adjusted to be consistent with the wirelength scoring strategy proposed below. Smaller scoring value now corresponds to a higher priority.

Regarding the wirelength score, a straightforward strategy is to place the block that yields the smallest wirelength increment relative to the current floorplan. However, this approach tends to favor blocks with fewer connections in the early stages while deferring the placement of highly connected blocks, which is undesirable. To mitigate this bias, we introduce the concept of *average wirelength*, which incorporates both the incremental cost and the connection density of a block. The definition is given in Eq. (11), where B_p represents the current set of placed blocks, and $n_{ij} = 1$ if block b_j and block b_i to be placed belong to the same net; otherwise, $n_{ij} = 0$. The average wirelength reflects the closeness between blocks, where a smaller value indicates a

Algorithm 4 Greedy Construction Algorithm for Fixed-Width Floorplanning

Require: A bounding box width W and a partial floorplan F_p.
Ensure: The resulting floorplan F_c and its objective value O_c.
$B_u \leftarrow$ the set of unplaced blocks in F_p
$S_p \leftarrow$ the current skyline of F_p
$F_c \leftarrow F_p$
while $B_u \neq \varnothing$ **do**
  $s_{bl} \leftarrow$ the bottom-left space of S_p
  while no block in B_u fits s_{bl} **do**
    Fill up s_{bl} and update S_p
    $s_{bl} \leftarrow$ the updated bottom-left space of S_p
  end while
  $b^* \leftarrow \arg\min_{b \in B_u} LocalEvaluation(b)$
  $S_p \leftarrow S_p \oplus b^*$ ▷ Update the skyline after placing b^*
  $F_c \leftarrow F_c \oplus b^*$ ▷ Update the floorplan after placing b^*
  $B_u \leftarrow B_u \setminus \{b^*\}$ ▷ Remove b^* from the set of unplaced blocks
end while
$O_c \leftarrow Cost(F_c)$
return (O_c, F_c)

higher priority for placement.

$$Wire\ Score(b_i) = \frac{\sum_{b_j \in B_p} n_{ij} \cdot \left(\left| x_i^c - x_j^c \right| + \left| y_i^c - y_j^c \right| \right)}{\sum_{b_j \in B_p} n_{ij}} \tag{11}$$

Since the area and wirelength scores are defined on different scales and are therefore not directly comparable in their raw forms, we adopt a ranking-based local evaluation strategy. This strategy is inspired by population management mechanisms in evolutionary algorithms for handling multiple evaluation criteria [41]. It focuses on the relative quality of candidates under each metric and avoids direct numerical comparison between heterogeneous measures. In detail, the candidate child nodes are sorted in ascending order according to their area and wirelength scores, respectively. The area and wirelength ranks of a child node c are denoted as $Rank_a(c)$ and $Rank_w(c)$. For each metric, if multiple child nodes obtain identical scores, ties are broken randomly to enhance diversification. Then, the local evaluation score of a child node c is defined as the weighted sum of its area and wirelength rankings, as shown in Eq. (12). The weighting parameter ξ is consistent with that used in the overall cost function (9), ensuring that the score reflects the same trade-off between area and wirelength as the global objective. Smaller local evaluation scores then indicate more promising branches.

$$Local\ Evaluation(c) = \xi \cdot Rank_a(c) + (1 - \xi) \cdot Rank_w(c). \tag{12}$$

4.3.2 Global evaluation

The global evaluation completes each partial solution using a greedy construction algorithm, as summarized in Algorithm 4. Given a width of the bounding box W and

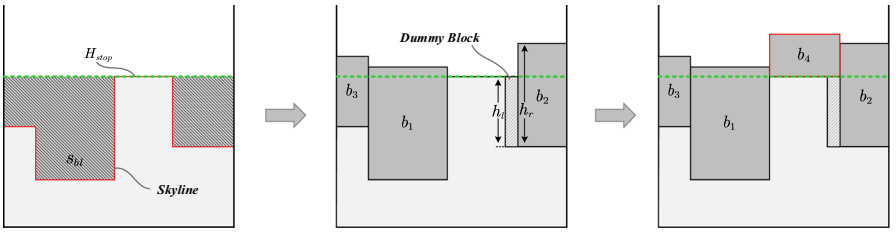

Fig. 4 An example of a greedy construction process for fixed-width floorplanning. **a** An initial partial solution. **b** Blocks b_1, b_2, and b_3 are placed sequentially, with a dummy block filling the narrow gap. **c** Block b_4 will not be placed when applying look-ahead evaluation because it exceeds the highest point H_{stop} of the initial skyline, as indicated by the green dotted line

a partial solution F_{p} with skyline S_{p}, the algorithm sequentially places the remaining unplaced blocks into the bottom-left space s_{bl} of the current skyline.

In each iteration, the local evaluation strategy is used to score all unplaced blocks, and the block with the minimum local evaluation score is selected and put into s_{bl} (line 10). Figure 4a, b shows an example of placing several blocks starting from a partial solution. If s_{bl} is a narrow gap that no remaining block can fit into, we fill it with a dummy block with the same width as s_{bl} and a height of $\min\{h_{\text{l}}, h_{\text{r}}\}$ (lines 6–9), as illustrated in Fig. 4b. We update the skyline S_p and the space s_{bl} once a block or dummy block is placed (lines 11–13). Finally, a complete floorplan along with its corresponding objective value is returned as the solution and score for the global evaluation.

4.3.3 Look-ahead evaluation

The construction process of the look-ahead evaluation is similar to that of the global evaluation but uses different stopping criteria. Instead of placing all blocks, it terminates when no block can be placed below the highest point of the initial skyline. Figure 4c illustrates this stopping condition, which aims to fill the shaded area in Fig. 4a as fully as possible while ensuring that the bottom of any newly placed block remains below the green dotted line H_{stop}. The scoring function for a partial solution is consistent with Eq. (9), but the computation of $A(F)$ and $L(F)$ only consider the placed blocks and the nets involved.

We introduce the look-ahead evaluation to compensate for the inaccuracy of the global evaluation, since long-term greedy construction may underestimate the potential of certain branches. Its effectiveness is validated through computational experiments in Sect. 5.6.

Table 1 Characteristics of the MCNC and GSRC datasets

Dataset	Instance	Blocks	Nets	I/O pad	Area
MCNC	apte	9	97	73	46.56
	xerox	10	203	2	19.35
	hp	11	83	45	8.83
	ami33	33	123	42	1.16
	ami49	49	408	22	35.45
GSRC	n30	30	349	212	0.209
	n50	50	485	209	0.199
	n100	100	885	334	0.180
	n200	200	1585	564	0.176
	n300	300	1893	569	0.273

5 Experimental results

We implemented the proposed solution framework integrating CWGA and IBSA in C++ (referred to as IBSA following) and conducted experiments on the MCNC[1] and GSRC[2] datasets on an Intel(R) Xeon(R) E5-2698v3 2.30 GHz CPU[3] and 192 GB RAM. The MCNC dataset comprises small instances with no more than 50 blocks, while the GSRC instances include up to 300 blocks. Table 1 gives the detailed statistics of all the tested instances. Columns *Blocks* and *Nets* represent the number of blocks and nets, respectively. The column *I/O pad* indicates the number of terminals connecting the boundary to the blocks, and the column *Area* denotes the total area (in mm^2) of all blocks.

5.1 Parameter analysis and settings

The parameters for IBSA are uniformly set based on preliminary experiments, and the tested values are detailed in Table 2. In the problem transformation stage, three parameters are involved in generating and selecting candidate widths, namely ρ, τ, and n_w. Parameters ρ and τ determine the lower and upper bounds of the candidate width range. An overly large ρ may exclude potentially suitable widths, whereas an overly small ρ may introduce extreme widths that waste runtime without improving solution quality. Similarly, τ is chosen to avoid excessively wide layouts that are unlikely to be competitive. The parameter n_w limits the number of promising candidate widths retained for downstream optimization. It trades off solution quality against search efficiency, where a larger value improves coverage at higher cost. In our implementation, ρ, τ, and n_w are set to 0.5, 1.2, and 400, respectively.

The improved beam search framework involves two control parameters. The parameter γ_u imposes an upper bound on the beam width to limit the maximum search effort

[1] https://s2.smu.edu/~manikas/Benchmark.html.

[2] http://vlsicad.eecs.umich.edu/BK/GSRCbench/HARD/.

[3] The single-thread performance is 1942 MOps/Sec according to https://www.cpubenchmark.net.

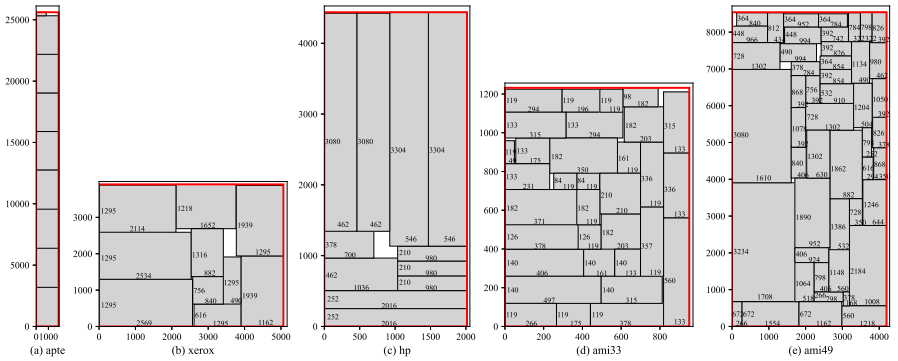

Fig. 5 Optimized floorplans corresponding to the MCNC benchmark areas reported for IBSA in Table 3

per subproblem. A smaller γ_u typically leads to faster convergence, while a larger γ_u can strengthen search capability and potentially yield better solutions. In line with our design goal of avoiding excessive time spent on a single subproblem, we set γ_u to a sufficiently large value based on empirical observations. In our implementation, the maximum beam width γ_u is set to 8192. The filter width β controls the number of branches in each beam level which undergo look-ahead and global evaluations, and it is set to twice the adaptive beam width γ. The performance impact of different β/γ ratios is further discussed in Sect. 5.6.

The weighting coefficient ξ in the objective function balances the area and wirelength metrics and is determined by the experimental scenario. Specifically, $\xi = 1$ is used when minimizing only the area, $\xi = 0$ when minimizing only the wirelength, and $\xi = 0.5$ when jointly optimizing both objectives. In addition, in the fixed-outline scenario, the aspect ratio R for each benchmark is set to 1.0, 2.0, or 3.0, and the maximum deadspace ratio D is fixed at 10%, consistent with the algorithms used for comparison.

5.2 Area optimization

In this section, we evaluate the proposed IBSA under the configuration that optimizes only the area (i.e., $\xi = 1.0$). First, we compare IBSA with various optimization algorithms under the free-outline scenario on the MCNC and GSRC benchmarks, including GA [19], RL [29], GARL [30], PSO [20], VOAS [26], PGHA [22], AHMA [23], ISSAA [18], EMA [25], and DPAHMA [24]. To the best of our knowledge, EMA and DPAHMA achieve the state-of-the-art results, with the former exhibiting higher optimization efficiency and the latter providing better area performance.

The experimental statistics on the MCNC benchmarks are presented in Table 3, where the column *Area* denotes the area of the optimized floorplan bounding box, and column *Time* reports the runtime of the different algorithms in seconds. We report the average area and runtime of the proposed algorithm over 50 independent runs. The last row of the table shows the relative standard deviations (RSDs) of these results to illustrate the stability of the algorithm. Since the compared algorithms were executed

Table 2 Parameter settings of our IBSA

Parameter	Value	Tested values	Description
ρ	0.5	$\{0.1, 0.3, 0.5, 0.7, 0.9\}$	Coefficient to control the minimum width
τ	1.2	$\{1.0, 1.2, 1.4, 1.6, 1.8\}$	Coefficient to control the maximum width
β	$2 \cdot \gamma$	$\{k \cdot \gamma \mid k = 1, 2, 4, 8\}$	Filter width for the tree search
n_w	400	$\{100, 200, 400, 800\}$	Maximum number of candidate widths
γ_u	8192	$\{2^k \mid k = 10, 11, 12, 13, 14, 15\}$	Maximum beam width for the tree search

Table 3 Comparison of results on MCNC benchmarks for area optimization

Algorithm	apte		xerox		hp		ami33		ami49	
	Area	Time	Area	Time	Area	Time	Area	Time	Area	Time
GA	46.92	8.8	20.0	20.8	9.03	16.8	1.22	1001.11	37.5	6222.2
RL	47.08	15.9	20.42	17.2	9.21	11.6	1.24	43.1	38.65	66.8
GARL	46.92	NR	20.09	NR	9.28	NR	1.18	NR	37.74	NR
PSO	46.92	NR	20.38	NR	NR	NR	1.29	NR	38.93	NR
VOAS	46.92	0.20	19.8	0.19	8.95	0.22	1.18	14.92	36.2	18.90
PGHA	46.92	NR	19.83	NR	NR	NR	1.20	NR	36.5	NR
AHMA	46.92	12.80	19.8	15.06	8.95	22.12	1.18	68.70	36.16	183.12
ISSAA	47.03	0.21	19.9	11.68	8.95	15.48	1.17	42.29	35.98	64.08
EMA	47.3	0.03	19.87	0.05	9.2	0.03	1.18	0.90	36.4	3.47
DPAHMA	46.92	NR	19.80	NR	8.95	NR	1.17	NR	36.04	NR
Our IBSA	46.92	0.06	19.80	0.20	8.95	0.15	1.17	0.58	35.87	5.29
RSD	0.00%		0.00%		0.00%		0.21%		0.11%	

NR: The results are not reported in the corresponding algorithm

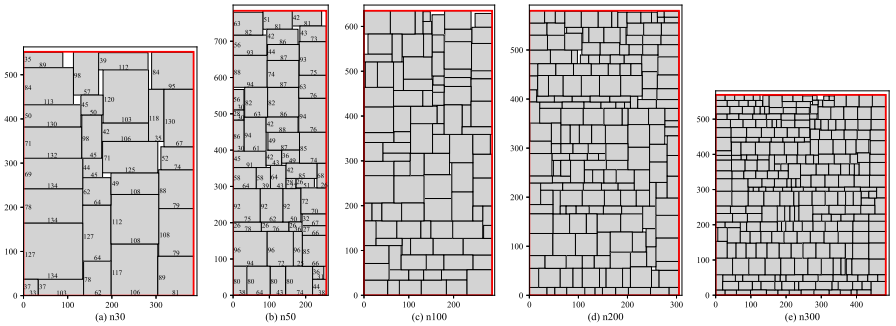

Fig. 6 Optimized floorplans corresponding to the GSRC benchmark areas reported for IBSA in Table 4

on different hardware platforms, we normalized the reported runtimes in the literature according to the corresponding CPU single-thread performance.[4] Note that the hardware specifications for GA and RL were not provided in the references, so their runtimes are presented as originally reported.

The entries marked "NR" denote that the respective results were not reported in the reference. For DPAHMA, only the cutoff time was reported, which was 180 s for apte, xerox, and hp and 1800s for the others. Therefore, the corresponding entries are also marked as "NR" in the tables. As shown in Table 3, the proposed IBSA achieves the smallest floorplan area for all instances compared with other algorithms and further improves the best-known upper bound on the area of the well-studied instance ami49. Compared with EMA, our method reduces deadspace by 0.82%, 0.07%, 2.83%, 0.86%, and 1.49% on the apte, xerox, hp, ami33, and ami49 instances, respectively, with comparable runtime.

It matches ISSAA on apte, hp, and ami33, while further reducing deadspace by 0.52% and 0.31% on xerox and ami49, respectively. Compared with DPAHMA, it achieves the same area on apte, xerox, hp, and ami33 in shorter runtime and yields an additional 0.47% deadspace reduction on the larger ami49 instance. Notably, our results on the apte, xerox, and hp instances also match the optimal area reported in the literature [4], which was proven to be optimal by the branch-and-bound method. Regarding solution stability, IBSA consistently reaches the best-known solutions on small-scale instances, including apte, xerox, and hp. For the larger instances ami33 and ami49, the relative standard deviations are 0.21% and 0.11%, respectively, suggesting highly robust performance with low run-to-run variability. Figure 5 illustrates the floorplans corresponding to the areas reported in Table 3, where the red box indicates the boundary of the floorplan and the size of each block is labeled along its respective side.

Table 4 compares the optimization results on the GSRC benchmark instances.

Algorithms that did not report results on these instances, including GA, PSO, PGHA, and AHMA, are excluded from this comparison. The proposed IBSA signifi-

[4] According to https://www.cpubenchmark.net, the normalization was performed based on the single-thread performance (MOps/Sec) of the CPUs on which the algorithms were executed.

Table 4 Comparison of results on GSRC benchmarks for area optimization

Algorithm	n30		n50		n100		n200		n300	
	Area	Time	Area	Time	Area	Time	Area	Time	Area	Time
RL	NR	NR	NR	NR	0.195	389.4	0.215	784.9	0.34	3766.9
GARL	0.219	3600	0.209	7200	0.196	10,800	0.198	18,000	0.315	25,200
VOAS	0.216	14.05	0.204	21.13	0.189	33.18	0.188	79.71	0.295	1115.59
ISSAA	NR	NR	NR	NR	0.183	67.33	0.18	224.58	0.28	701.65
EMA	0.216	0.63	0.207	3.40	0.19	28.57	0.187	93.56	0.28	259.95
DPAHMA	NR	NR	NR	NR	0.184	NR	0.183	NR	0.291	NR
Our IBSA	0.213	7.05	0.201	8.26	0.182	6.77	0.177	15.74	0.275	35.24
RSD	0.12%		0.11%		0.09%		0.06%		0.03%	

NR: The corresponding results are not reported in the reference

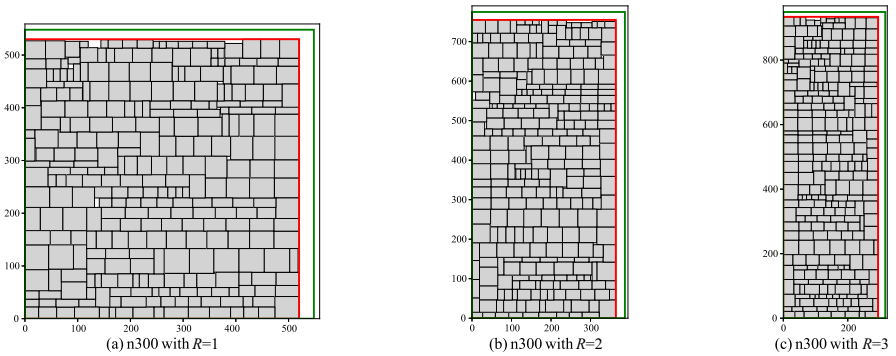

Fig. 7 Optimized floorplans for the n300 instance with aspect ratios $R = 1, 2, 3$, where the green box denotes the fixed-outline constraint

cantly improves the best-known floorplan area on the GSRC benchmarks, particularly for large-scale instances.

Compared with EMA, IBSA reduces the deadspace by 1.44%, 3.02%, 4.46%, 5.69%, and 1.83% on the n30, n50, n100, n200, and n300 instances, respectively. Compared with DPAHMA, IBSA further reduces the deadspace by 1.24%, 3.69%, and 5.79% on the n100, n200, and n300 instances, respectively. The relative standard deviations further indicate that IBSA is more stable on large-scale instances than on small-scale ones, potentially attributable to the greater diversity of compact layout combinations in larger circuits. Figure 6 shows the floorplans corresponding to the reported areas.

The runtime comparison is also notable. IBSA completes the largest instance n300 within one minute on average, whereas the competing methods reported in Table 4 require substantially longer runtimes on this instance. We believe that this advantage stems from our problem transformation and the beam search algorithm, which consistently explores a number of high-quality partial floorplans. In contrast, optimization methods based on tree structures or sequence representations must reconstruct the entire floorplan after each move for evaluation, which can be time-consuming for large-scale instances.

We then conducted area optimization comparisons under various fixed-outline constraints on the GSRC benchmarks. For each instance, we performed 50 independent runs under aspect ratio constraints $R = 1, 2, 3$, with a runtime limit of one minute per run. We compared our results against the state-of-the-art algorithms SA-EA [15] and EMA [25], with SA-EA including three evaluation function variants, namely SA-EA1, SA-EA2, and SA-EAL. The other compared methods do not report results under this setting and thus are not included in this comparison. Table 5 presents the comparative results of the different algorithms in terms of success rate (SR) and average area (AA, in μm^2). The last column reports the relative standard deviations of the optimized results under different configurations. It can be observed that all compared algorithms successfully solve the problem under the fixed-outline constraints, while IBSA consistently yields smaller floorplan areas for all instances. Compared with SA-EAL, IBSA yields average improvements of 1.78%, 1.39%, 2.78%, 4.28%, and 5.06% on

Table 5 Comparison of GSRC benchmark results under fixed-outline constraints for area optimization

Instance	R	SA-EA1		SA-EA2		SA-EAL		EMA		Our IBSA		RSD
		SR	AA	SR	AA	SR	AA	SR	AA	SR	AA	
n30	1	100	220,942	100	217,120	100	217,592	100	219,430	100	213,648	0.20%
	2	100	219,854	100	217,365	100	216,464	100	226,128	100	212,982	0.17%
	3	100	220,911	100	217,008	100	216,529	100	227,425	100	212,827	0.17%
n50	1	100	207,708	100	205,660	100	204,295	100	206,080	100	203,193	0.22%
	2	100	209,538	100	206,032	100	205,328	100	204,728	100	201,822	0.14%
	3	100	207,876	100	205,556	100	205,352	100	210,380	100	201,656	0.12%
n100	1	100	190,207	100	188,320	100	187,880	100	186,560	100	182,668	0.15%
	2	100	189,616	100	187,200	100	187,200	100	185,742	100	181,975	0.14%
	3	100	191,550	100	187,170	100	186,624	100	190,750	100	181,587	0.13%
n200	1	100	187,295	100	187,085	100	185,736	100	181,332	100	177,467	0.10%
	2	100	186,942	100	186,067	100	184,450	100	180,290	100	177,153	0.10%
	3	100	186,001	100	186,136	100	183,931	100	183,995	100	176,923	0.06%
n300	1	100	294,022	100	292,968	100	290,485	100	283,544	100	275,419	0.08%
	2	100	290,729	100	291,412	100	288,702	100	280,962	100	274,829	0.08%
	3	100	291,259	100	290,880	100	287,244	100	308,000	100	274,723	0.06%

Table 6 Wirelength optimization results on the GSRC benchmarks

Instance	R	Average wirelength				
		SA-EA1	SA-EA2	SA-EAL	EMA	Our IBSA (RSD)
n30	1	119,402	119,402	115,111	NR	108,809 (0.86%)
	2	125,393	124,548	116,585	NR	115,628 (0.55%)
	3	131,808	135,316	123,715	NR	126,236 (0.68%)
n50	1	152,894	156,066	147,249	NR	121,516 (1.34%)
	2	160,789	162,782	150,668	NR	128,211 (1.04%)
	3	172,382	174,051	161,605	NR	140,403 (1.12%)
n100	1	240,946	245,880	236,875	206,218	191,595 (1.42%)
	2	258,880	269,545	238,797	229,245	201,746 (1.03%)
	3	305,821	279,266	246,404	256,614	225,493 (1.23%)
n200	1	450,941	456,738	433,367	373,232	389,343 (0.89%)
	2	468,728	467,049	455,293	402,566	406,548 (0.78%)
	3	499,649	500,965	474,661	424,836	448,414 (1.00%)
n300	1	639,518	633,150	613,523	512,465	515,642 (0.55%)
	2	667,876	671,823	646,088	535,824	575,464 (1.23%)
	3	719,196	721,808	677,303	560,365	657,416 (1.10%)

NR: The corresponding results are not reported in the reference

the n30, n50, n100, n200, and n300 instances, respectively. Compared with EMA, the corresponding improvements are 5.36%, 2.44%, 3.12%, 2.67%, and 5.80%. Moreover, IBSA achieves higher fill ratios under high aspect ratio constraints, as less deadspace tends to occur at the floorplan boundaries, whereas SA-EA1 and EMA deteriorate under the $R = 3$ setting.

Beyond solution quality, IBSA also exhibits high stability under different fixed-outline constraints. It consistently achieves low relative standard deviations across runs, because these constraints only affect candidate width generation and do not change the underlying search structure. Figure 7 depicts the floorplans of IBSA on the n300 instance for $R = 1, 2, 3$, with deadspace ratios of 0.89%, 0.60%, and 0.52% for the three cases. In conclusion, our proposed algorithm shows obvious advantages on large instances and area optimization of floorplans.

5.3 Wirelength optimization

We performed wirelength optimization with the objective weight coefficient set to $\xi = 0$, under aspect ratio constraints of $R = 1, 2, 3$ and a deadspace ratio of $D = 10\%$, conducting 50 independent runs for each GSRC benchmark instance with a one-minute time limit. Table 6 reports the average wirelengths against SA-EA (including its three variants) and EMA. We also report the relative standard deviations (RSDs) under different configurations to demonstrate the algorithmic stability. The success rate is not reported in the table, as all algorithms achieved complete success in solving the problem under the fixed-outline constraint. Compared with SA-EA, IBSA achieves average

wirelength reductions of 3.15%, 15.17%, 14.37%, 8.80%, and 9.94% on the n30, n50, n100, n200, and n300 instances, respectively, with a slight disadvantage of 2.04% on the n30 instance at an aspect ratio of $R = 3$. Compared with EMA, IBSA reduces wirelength by 10.40% on the n100 instance. Nevertheless, for the larger instances n200 and n300, it exhibits a deterioration, being 3.62% and 8.45% worse than EMA, respectively. Compared with pure area optimization, wirelength optimization exhibits slightly higher variability across runs. Nevertheless, the relative standard deviations remain within 0.55% to 1.34%, indicating robust and consistent performance.

5.4 Area and wirelength optimization

In this experiment, we set weight $\xi = 0.5$ to balance the optimization between floorplan area and wirelength. The proposed IBSA is compared with state-of-the-art algorithms reported in the literature, including PSO [20], HSA [17], ISSAA [18], PGHA [22], AHMA [23], LOA [27], VOAS [26], ASSA [16], DeFer [28], EMA [25], and DPAHMA [24]. We exclude the RL-based methods RL [29] and GARL [30] because the literature does not specify the wirelength computation used in their evaluations. The first six algorithms reported results only on the MCNC benchmark set, whereas ASSA and DeFer reported results only for the GSRC benchmark set. We conduct 50 independent runs of IBSA on each instance and report the average results together with the corresponding relative standard deviations. Table 7 summarizes the area and wirelength results for the MCNC benchmarks, whereas Table 8 summarizes the corresponding results for the GSRC benchmarks. Table 9 further reports the runtime comparison corresponding to Tables 7 and 8. The runtimes of the compared algorithms were scaled according to the hardware performance on which they were run, and thus may differ from the original values reported in the references. Runtime is unavailable for PSO, PGHA, and LOA in the referenced papers and is therefore not listed in Table 9. As noted above, DPAHMA reports only cutoff times for runtime rather than measured runtimes and is thus not included in Table 9.

Due to the multi-objective nature of the floorplanning problem, it is challenging for one algorithm to dominate another one simultaneously in terms of both area and wirelength minimization. Intensive optimization of a single metric by an algorithm often comes at the expense of performance in the other metric. For example, compared with EMA, IBSA achieves area reductions of 2.51% and 1.64% on the xerox and ami33 instances, while wirelength increases by 2.50% and 6.44%, respectively. However, on the hp instance, IBSA reduces area and wirelength by 3.26% and 6.65%, respectively. On n200, the corresponding reductions are 3.62% and 9.80%. Compared with DPAHMA, IBSA consistently attains a smaller floorplan area and further reduces wirelength by 15.48% and 5.78% on the hp and n100 instances, respectively.

Notably, for the n30, n100, and n200 instances, IBSA produces shorter wirelengths than those obtained under wirelength-only optimization in Table 6, because the area objective offers auxiliary guidance for wirelength reduction and the relaxed boundary constraints allow greater layout flexibility. Compared with single-objective optimization, the relative standard deviations increase slightly during bi-objective optimization.

Table 7 Comparison of results on MCNC benchmarks for area and wirelength optimization

Algorithm	apte		xerox		hp		ami33		ami49	
	Area	WL	Area	WL	Area	WL	Area	WL	Area	WL
PSO	47.31	263	20.2	477	9.5	136	1.28	69	38.8	880
HSA	47.12	480	20.89	513	9.47	144.3	1.21	61.2	37.8	1020
ISSAA	48.47	408.6	20.42	387.6	9.40	153.1	1.26	49.28	37.76	824.5
PGHA	47.44	463	20.2	497	NR	NR	1.24	48.4	38.6	673
AHMA	48.48	408.7	20.42	399.6	9.49	153.4	1.24	47.71	38.94	666.3
LOA	48.03	316.5	20.5	337	9.89	143	1.23	52.22	38.45	822.7
VOAS	47.1	245.9	20.3	379.6	9.46	149.8	1.22	59.48	37.8	667
EMA	47.56	333.9	19.99	460.2	9.33	139.8	1.22	44.92	38.48	627.8
DPAHMA	48.48	408.66	20.42	387.65	9.41	154.4	1.23	47.79	38.82	678.06
Average	47.78	369.81	20.37	426.53	9.49	146.7	1.24	53.33	38.38	762.15
Our IBSA	47.31	469.82	19.94	471.51	9.20	130.5	1.20	57.51	37.51	757.38
RSD	0.85%	0.92%	0.31%	1.13%	0.73%	0.51%	1.16%	1.85%	1.89%	3.00%

NR: The corresponding results are not reported in the reference

Table 8 Comparison of results on GSRC benchmarks for area and wirelength optimization

Algorithm	n30		n50		n100		n200		n300	
	Area	WL	Area	WL	Area	WL	Area	WL	Area	WL
VOAS	0.221	95.46	0.215	181.1	0.208	212.9	0.197	438.1	0.307	606.3
ASSA	NR	NR	0.215	135.1	0.205	209.8	0.192	404.5	0.300	556.9
DeFer	NR	NR	NR	NR	0.191	209.8	0.187	374.6	0.291	503.3
EMA	NR	NR	0.214	133.2	0.201	165.3	0.193	411.3	0.310	436.4
DPAHMA	NR	NR	NR	NR	0.192	197.1	0.190	350.4	0.300	532.1
Average	0.221	95.46	0.215	149.8	0.199	199.0	0.192	395.8	0.302	525.2
Our IBSA	0.218	106.3	0.211	135.6	0.192	185.7	0.186	371.0	0.289	555.4
RSD	0.83%	1.20%	0.80%	1.44%	0.80%	1.23%	0.64%	0.79%	0.56%	0.82%

NR: The corresponding results are not reported in the reference

Table 9 Runtime comparison corresponding to Tables 7 and 8 on MCNC and GSRC benchmarks for area and wirelength optimization

Algorithm	MCNC time					GSRC time				
	apte	xerox	hp	ami33	ami49	n30	n50	n100	n200	n300
HSA	3.74	14.74	15.15	28.72	89.68	NR	NR	NR	NR	NR
ISSAA	52.64	103.7	62.94	63.34	129.2	NR	NR	NR	NR	NR
AHMA	14.45	28.73	33.70	143.6	398.5	NR	NR	NR	NR	NR
VOAS	0.37	0.40	0.63	25.77	29.47	22.25	32.81	48.94	94.91	1147
EMA	0.57	0.63	0.67	1.79	7.40	NR	1.79	26.72	82.52	251.0
ASSA	NR	NR	NR	NR	NR	NR	7.9	26.6	102.5	216.2
DeFer	NR	NR	NR	NR	NR	NR	NR	0.10	0.22	0.28
Average	14.36	29.64	22.62	52.64	130.8	22.3	17.30	25.3	59.22	466.3
Our IBSA	12.64	15.61	21.13	26.65	32.70	13.3	42.2	41.8	29.4	44.5

NR: The corresponding results are not reported in the reference

This can be attributed to minor run-to-run differences in the trade-off between the two objectives, yet the overall variability remains low.

We also present the average area and wirelength of other comparative algorithms for an overall assessment. On the GSRC benchmarks, IBSA reduces area relative to the average by 1.36%, 1.71%, 3.71%, 3.02%, and 4.18% on the n30, n50, n100, n200, and n300 instances, respectively, while the corresponding changes in wirelength are 11.36%, −9.48%, −6.66%, −6.26%, and 5.75%. In summary, the proposed IBSA produces smaller chip areas with wirelengths comparable to those of other algorithms. In terms of runtime, IBSA explores a more diverse search space to obtain floorplans that balance area and wirelength and consequently requires more time than single-objective floorplanning optimization.

Nevertheless, IBSA retains competitive overall efficiency and consistently produces the reported results within the one-minute time limit, with more favorable runtimes on the larger instances.

5.5 Impact of adaptive width selection

We adopt an adaptive width selection approach (AWSA) to probabilistically explore the transformed fixed-width floorplanning subproblems based on their past performance. To verify the effectiveness of this strategy, we conduct an ablation study by comparing AWSA with a randomized variant, namely the random width selection approach (RWSA), which chooses candidate widths uniformly at random.

We perform Mann–Whitney U tests on the final area results of AWSA and RWSA, as reported in Table 10. All experiments are conducted with $\xi = 1$, where the objective reduces to minimizing floorplan area only. For each instance, both methods are executed for 50 independent runs with a one-minute time limit per run. For each instance, the null hypothesis H_0 states that there is no statistically significant difference between AWSA and RWSA in the distributions of final areas, whereas the

Table 10 Mann–Whitney statistical test results on area optimization for AWSA and RWSA

Instances	AWSA			RWSA			U	p value*
	n	Mean rank	Rank sum	n	Mean rank	Rank sum		
apte	50	50.50	2525.00	50	50.50	2525.00	1250.0	1.000000
xerox	50	50.50	2525.00	50	50.50	2525.00	1250.0	1.000000
hp	50	50.50	2525.00	50	50.50	2525.00	1250.0	1.000000
ami33	50	49.35	2467.50	50	51.65	2582.50	1192.5	0.688807
ami49	50	45.70	2285.00	50	55.30	2765.00	1010.0	0.097000
n30	50	43.54	2177.00	50	57.46	2873.00	902.0	0.016196
n50	50	45.52	2276.00	50	55.48	2774.00	1001.0	0.086605
n100	50	42.33	2116.50	50	58.67	2933.50	841.5	0.004902
n200	50	40.41	2020.50	50	60.59	3029.50	745.5	0.000507
n300	50	41.23	2061.50	50	59.77	2988.50	786.5	0.001364

*Statistical significance is assessed at the 0.05 level. Values with $p < 0.01$ are considered highly significant

alternative hypothesis H_1 states that the two methods differ significantly. Since the test is rank-based, a smaller mean rank indicates a tendency toward smaller area and thus better performance.

As shown in Table 10, the two methods produce identical outcomes on the small instances apte, xerox, and hp, indicating that both strategies can reliably reach the best solutions within the given budget on relatively small search spaces. As the instances become larger, the search space expands and the choice of widths has a stronger impact under the same time limit. AWSA yields statistically significant improvements on n30 with a p value below 0.05 and highly significant improvements on n100, n200, and n300 with p values below 0.01, while showing a favorable trend on ami49 and n50. These results indicate that the adaptive strategy can better concentrate the search effort on more promising widths under a fixed time budget, leading to improved final solution quality with statistical support.

5.6 Effectiveness of improved beam search

Beam search is a tree search framework that proceeds without backtracking, which makes its performance highly sensitive to the pruning strategy employed. We conduct ablation experiments on multiple variants of the complete IBSA to evaluate the contribution of the proposed pruning strategy:

- IBSA-NOLE: No look-ahead evaluation is used in the pruning strategy, which makes the algorithm behave like the classical beam search framework.
- IBSA-NOGE: No global evaluation is used in the pruning strategy, which means only selecting promising branches by look-ahead evaluation.
- IBSA-$k\gamma$: Applying different ratios of the filter width β to the beam width γ in the pruning strategy, i.e., $\beta = k \cdot \gamma$. Here, k takes values of 1, 4, and 8, where $k = 1$ indicates selecting γ branches based solely on local evaluation. For IBSA, the value of k is set to 2.

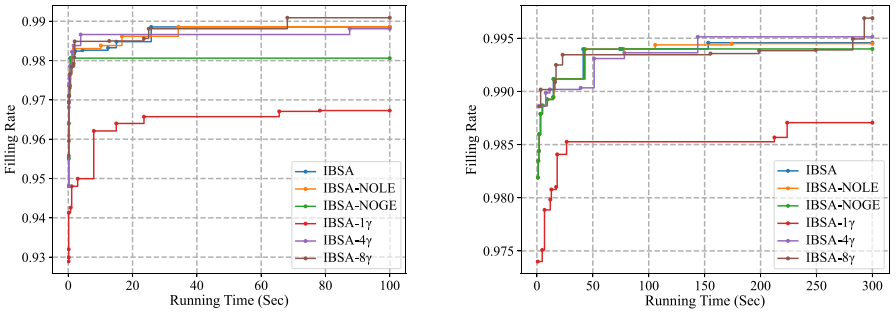

Fig. 8 Area filling ratio over runtime for different IBSA variants on the ami49 and n300 instances

We conduct experiments on the IBSA algorithm and its five variants on the largest-scale instances ami49 and n300 of the MCNC and GSRC datasets, respectively, considering that dataset characteristics and strategy differences can be better reflected on large-scale instances. We set the area objective weight to $\xi = 1$ with an extended runtime for testing. Then, the performance of the algorithm can be measured by the filling ratio of the floorplan, where a higher value indicates better performance, thereby avoiding an ambiguous bi-objective comparison. Figure 8 shows the variation of the filling ratio over time for each variant algorithm under this configuration.

IBSA-NOLE significantly outperforms IBSA-NOGE on instance ami49, whereas their performance becomes comparable on the larger instance n300. This demonstrates that greedy construction-based global evaluation can provide a reasonable estimation of optimization potential for partial solutions when the number of blocks is small but becomes less accurate as the number of blocks increases. Look-ahead evaluation compensates for the limitations of global evaluation by assessing results only for a near future period. IBSA provides a higher filling rate than IBSA-NOLE on instance n300 and improves the optimization efficiency on instance ami49. For a more detailed comparison between IBSA and its ablated variant IBSA-NOLE, we further conduct Mann–Whitney U tests on area optimization, as reported in Table 11. For each instance, the null hypothesis H_0 states that there is no statistically significant difference in the floorplan area between IBSA and IBSA-NOLE, whereas the alternative hypothesis H_1 states that their area results differ significantly. Since the test is performed on area values, a lower mean rank indicates better performance. The results show that IBSA and IBSA-NOLE produce identical outcomes on apte, xerox, and hp. IBSA achieves statistically significant improvements on ami33, ami49, n200, and n300 at the 0.01 level, while the differences on n30 and n50 are marginal and not significant at the 0.05 level. On n100, no significant difference is observed, and IBSA shows a lower mean rank. Overall, IBSA tends to obtain smaller areas than IBSA-NOLE on the instances where the difference is significant, indicating that the look-ahead evaluation contributes to improved optimization.

The ratio of the filtering width β to the beam width γ determines the trade-off between local evaluation and long-term evaluation. As shown in Fig. 8, IBSA-1γ selects promising branches based only on local evaluations and has a clear gap com-

Table 11 Mann–Whitney statistical test results on area optimization for IBSA and IBSA-NOLE

Instances	IBSA			IBSA-NOLE			U	p value*
	n	Mean rank	Rank sum	n	Mean rank	Rank sum		
apte	50	50.50	2525.00	50	50.50	2525.00	1250.0	1.000000
xerox	50	50.50	2525.00	50	50.50	2525.00	1250.0	1.000000
hp	50	50.50	2525.00	50	50.50	2525.00	1250.0	1.000000
ami33	50	42.48	2124.00	50	58.52	2926.00	849.0	0.004900
ami49	50	42.29	2114.50	50	58.71	2935.50	839.5	0.004663
n30	50	44.99	2249.50	50	56.01	2800.50	974.5	0.057671
n50	50	45.09	2254.50	50	55.91	2795.50	979.5	0.062526
n100	50	47.68	2384.00	50	53.32	2666.00	1109.0	0.332668
n200	50	41.73	2086.50	50	59.27	2963.50	811.5	0.002511
n300	50	41.99	2099.50	50	59.01	2950.50	824.5	0.003320

*Statistical significance is assessed at the 0.05 level. Values with $p < 0.01$ are considered highly significant

pared to other variants because it lacks a long-term view. As the filter width increases, the search tends to yield better results. IBSA-4γ performs similarly to the complete IBSA, with a slightly worse result on instance ami49 and a slightly better result on instance n300. IBSA-8γ achieves the best filling rate on both instances among all algorithm variants because it evaluates more candidate branches from a global perspective. However, this comes at the cost of a longer search time.

6 Conclusion

This paper presents a reduction framework for VLSI floorplanning, where a candidate width generation algorithm (CWGA) reformulates the original problem into a series of fixed-width subproblems for systematic exploration. An improved beam search algorithm (IBSA) is then proposed to solve fixed-width floorplanning subproblems efficiently. The proposed IBSA integrates a tailored branching mechanism with three complementary evaluation strategies, namely local, look-ahead, and global evaluations, to achieve a better balance between exploration and pruning in the search process. Comprehensive experiments are conducted to analyze the effects of different components, including the pruning scheme and beam width control, on the overall performance of the algorithm. Experimental results on the classical MCNC and GSRC benchmarks demonstrate that IBSA achieves highly competitive results compared with state-of-the-art algorithms for both free-outline and fixed-outline floorplanning, especially on large-scale instances. Future work will focus on further wirelength optimization within the proposed framework.

Author contribution C.L. and Y.L. conceived the study and developed the methodology. C.L., Y.Z., and Y.L. performed the formal analysis and investigation. Z.S. and J.D. secured the funding. C.L. provided resources and performed visualization. C.L. and Y.Z. prepared the original draft. Z.L. and Z.S. supervised the study. All authors reviewed, edited, and approved the final manuscript.

Funding This work was supported in part by the Innovation Program for Quantum Science and Technology under Grant No. 2024ZD0300500, the National Natural Science Foundation of China (NSFC) under Grant Nos. 62402191 and 62572210, and the Interdisciplinary Research Program of HUST under Grant No. 5003300129.

Data availability The benchmark datasets used in this study are publicly available within the manuscript, and the results will be made available on request.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Lee W-P, Liu H-Y, Chang Y-W (2006) Voltage island aware floor planning for power and timing optimization. In: Proceedings of the 2006 IEEE/ACM International Conference on Computer-Aided Design, pp 389–394
2. Dayasagar Chowdary S, Sudhakar M (2024) Linear programming-based multi-objective floorplanning optimization for system-on-chip. J Supercomput 80(7):9663–9686. https://doi.org/10.1007/s11227-023-05812-0
3. Lai M, Wong D (2001) Slicing tree is a complete floorplan representation. In: Proceedings Design, Automation and Test in Europe. Conference and Exhibition 2001, IEEE, pp 228–232
4. Chan HH, Markov IL (2004) Practical slicing and non-slicing block-packing without simulated annealing. In: Proceedings of the 14th ACM Great Lakes Symposium on VLSI, pp 282–287
5. Adya SN, Markov IL (2001) Fixed-outline floorplanning through better local search. In: Proceedings 2001 IEEE International Conference on Computer Design: VLSI in Computers and Processors. ICCD 2001, IEEE, pp 328–334
6. Wong D, Liu C (1986) A new algorithm for floorplan design. In: 23rd ACM/IEEE Design Automation Conference, IEEE, pp 101–107
7. Young FY, Wong D (1998) Slicing floorplans with pre-placed modules. In: Proceedings of International Conference on Computer-Aided Design, pp 252–258
8. Guo P-N, Cheng C-K, Yoshimura T (1999) An O-tree representation of non-slicing floorplan and its applications. In: Proceedings 1999 Design Automation Conference (Cat. No. 99CH36361), pp 268–273
9. Chang Y-C, Chang Y-W, Wu G-M, Wu S-W (2000) B*-trees: a new representation for non-slicing floorplans. In: Proceedings 37th Design Automation Conference, pp 458–463. https://doi.org/10.1109/DAC.2000.855354
10. Lin J-M, Hung Z-X (2011) Skb-tree: a fixed-outline driven representation for modern floorplanning problems. IEEE Trans Very Large Scale Integr (VLSI) Syst 20(3):473–484
11. Ma Y, Dong S, Hong X, Cai Y, Cheng C-K, Gu J (2001) VLSI floorplanning with boundary constraints based on corner block list. In: Proceedings of the 2001 Asia and South Pacific Design Automation Conference, pp 509–514
12. Lin J-M, Chang Y-W (2001) TCG: a transitive closure graph-based representation for non-slicing floorplans. In: Proceedings of the 38th Annual Design Automation Conference, pp 764–769
13. Tang X, Wong D (2001) FAST-SP: a fast algorithm for block placement based on sequence pair. In: Proceedings of the 2001 Asia and South Pacific Design Automation Conference, pp 521–526
14. Gwee B-H, Lim M-H (1999) A GA with heuristic-based decoder for ic floorplanning. Integration 28(2):157–172
15. Huang Z, Lin Z, Zhu Z, Chen J (2020) An improved simulated annealing algorithm with excessive length penalty for fixed-outline floorplanning. IEEE Access 8:50911–50920
16. Xu Q, Chen S, Li B (2016) Combining the ant system algorithm and simulated annealing for 3D/2D fixed-outline floorplanning. Appl Soft Comput 40:150–160
17. Chen J, Zhu W, Ali MM (2010) A hybrid simulated annealing algorithm for nonslicing VLSI floorplanning. IEEE Trans Syst Man Cybern Part C (Appl Rev) 41(4):544–553

18. Anand S, Saravanasankar S, Subbaraj P (2012) Customized simulated annealing based decision algorithms for combinatorial optimization in VLSI floorplanning problem. Comput Optim Appl 52(3):667–689
19. Tang M, Sebastian A (2005) A genetic algorithm for VLSI floorplanning using o-tree representation. In: Applications of Evolutionary Computing, 2005 Proceedings, Springer, pp 215–224
20. Chen G, Guo W, Chen Y (2010) A PSO-based intelligent decision algorithm for VLSI floorplanning. Soft Comput 14(12):1329–1337
21. Chen J, Chen G, Guo W (2009) A discrete pso for multi-objective optimization in VLSI floorplanning. In: Advances in Computation and Intelligence: 4th International Symposium, Springer, pp 400–410
22. Sivaranjani P, Senthil Kumar A (2015) Thermal-aware non-slicing VLSI floorplanning using a smart decision-making pso-ga based hybrid algorithm. Circuits Syst Signal Process 34(11):3521–3542
23. Chen J, Liu Y, Zhu Z, Zhu W (2017) An adaptive hybrid memetic algorithm for thermal-aware non-slicing VLSI floorplanning. Integration 58:245–252
24. Jiang L, Ouyang D, Zhou H, Tian N, Zhang L (2023) DPAHMA: a novel dual-population adaptive hybrid memetic algorithm for non-slicing vlsi floorplans. J Supercomput 79(14):15496–15534. https://doi.org/10.1007/s11227-023-05277-1
25. Shanthi J, Rani DGN, Rajaram S (2022) An enhanced memetic algorithm using skb tree representation for fixed-outline and temperature driven non-slicing floorplanning. Integration 86:84–97
26. Hoo C-S, Jeevan K, Ganapathy V, Ramiah H (2013) Variable-order ant system for VLSI multiobjective floorplanning. Appl Soft Comput 13(7):3285–3297
27. Shunmugathammal M, Columbus CC, Anand S (2020) A nature inspired optimization algorithm for VLSI fixed-outline floorplanning. Analog Integr Circ Sig Process 103(1):173–186
28. Yan JZ, Chu C (2010) Defer: deferred decision making enabled fixed-outline floorplanning algorithm. IEEE Trans Comput Aided Des Integr Circuits Syst 29(3):367–381
29. He Z, Ma Y, Zhang L, Liao P, Wong N, Yu B, Wong MD (2020) Learn to floorplan through acquisition of effective local search heuristics. In: 2020 IEEE 38th International Conference on Computer Design, IEEE, pp 324–331
30. Liu K, Gu J, Gu H, Zhu Z (2023) A hybrid reinforcement learning and genetic algorithm for VLSI floorplanning. In: Proceedings of the 2023 15th International Conference on Machine Learning and Computing, pp 412–418
31. Imahori S, Yagiura M, Ibaraki T (2005) Improved local search algorithms for the rectangle packing problem with general spatial costs. Eur J Oper Res 167(1):48–67
32. Huang W, Liu J-F (2006) A deterministic heuristic algorithm based on Euclidian distance for solving the rectangles packing problem. Chin J Comput 29(5):734
33. He K, Huang W, Jin Y (2012) An efficient deterministic heuristic for two-dimensional rectangular packing. Comput Oper Res 39(7):1355–1363
34. Burke EK, Kendall G, Whitwell G (2004) A new placement heuristic for the orthogonal stock-cutting problem. Oper Res 52(4):655–671
35. He K, Ji P, Li C (2015) Dynamic reduction heuristics for the rectangle packing area minimization problem. Eur J Oper Res 241(3):674–685
36. Wei L, Zhu W, Lim A, Liu Q, Chen X (2018) An adaptive selection approach for the 2d rectangle packing area minimization problem. Omega 80:22–30
37. Ow PS, Morton TE (1988) Filtered beam search in scheduling. Int J Prod Res 26(1):35–62
38. Shunmugathammal M, Christopher Columbus C, Anand S (2020) A novel B* tree crossover-based simulated annealing algorithm for combinatorial optimization in VLSI fixed-outline floorplans. Circuits Syst Signal Process 39:900–918
39. Wei L, Oon W-C, Zhu W, Lim A (2011) A skyline heuristic for the 2d rectangular packing and strip packing problems. Eur J Oper Res 215(2):337–346. https://doi.org/10.1016/j.ejor.2011.06.022
40. Bortfeldt A (2013) A reduction approach for solving the rectangle packing area minimization problem. Eur J Oper Res 224(3):486–496
41. Zhou Y, Hao J-K, Glover F (2018) Memetic search for identifying critical nodes in sparse graphs. IEEE Trans Cybern 49(10):3699–3712

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.